To implement a granular, graphically configurable Role-Based Access Control (RBAC) system that maintains synchronization between code and storage, a rigorous architecture is required.

Here is the most robust approach to achieve flexibility and Front/Back-end consistency.

1. Database Modeling (The Foundation)

To allow for graphical management, permissions must not be "hardcoded" solely in the code but must persist in the database.

The standard relational model should include 5 entities:

1. **Users:** The actual users.
2. **Roles:** Logical groups (e.g., Admin, Editor, Guest).
3. **Permissions:** Atomic functionalities (e.g., user.create, invoice.read).
4. **Role_Permissions:** Linking table (Which role has which permission).
5. **User_Roles:** Linking table (Which user has which role).

Key Point: The Permissions table is the source of truth for your graphical interface.

2. Naming and "Discovery" (Handling Updates)

This is where maintainability is determined when adding features. If you add a function in the code, it must automatically appear in the administration interface.

A. Naming Convention

Adopt a strict hierarchical structure using strings (Scoping).

Format: resource:action or module:resource:action.

- users:read
- users:write
- reports:export_pdf

B. Synchronization (Code First)

Never create permissions manually via SQL. Permissions should be defined in the code (Back-end) via decorators or constants, and then synchronized.

Synchronization Algorithm (at API startup):

1. Scan the code to find all routes/functions protected by a permission.

2. Extract the unique list of permissions (e.g., ['users:read', 'users:write']).
3. **Upsert** into the database:
    - If the permission does not exist in DB $\rightarrow$ Create the entry (so it appears in the UI).
    - If the permission exists in DB but is no longer in the code $\rightarrow$ Mark as "obsolete" or delete (with caution).

Benefit: As soon as you deploy a new version of the code with a new feature, the permission automatically appears in your role management interface.

3. Back-End Implementation (Security)

The Back-end should not concern itself with roles, but **only with permissions**.

- **Middleware / Guard:** Intercepts every request.
- **Logic:**
    1. Retrieve the current user.
    2. Load their roles $\rightarrow$ Load permissions associated with those roles.
    3. Flatten everything into an array of strings: ['users:read', 'products:write'].
    4. Check if the permission required by the endpoint (e.g., users:read) is present in this array.

Conceptual Example (Pseudo-code):

TypeScript

```
// Definition in the controller
@RequirePermission('invoices:validate')
function validateInvoice(id) { ... }


// In the Guard (Interceptor)
function canActivate(user, requiredPermission) {
  const userPermissions = db.getPermissionsForUser(user.id);
  return userPermissions.includes(requiredPermission);
```

}

4. Front-End Implementation (UX and Consistency)

The Front-end must receive the user's list of permissions immediately upon login (or via a /me endpoint).

- **Structural Directive (Vue/Angular/React):** Create a component or directive that conditions DOM rendering.
    - *Example:* <ShowIf permission="users:create"><Button>Add</Button></ShowIf>
- **Route Guards:** Prevent navigation to a URL if the permission is missing.

Important: Front-end hiding is only cosmetic. Real security is ensured by rejecting the request on the Back-end (point 3).

5. The Graphical Management Interface (Configuration)

To make this graphically configurable, the ideal interface is a **Permission Matrix**.

How the Admin Screen works:

1. **Columns:** Display existing Roles (retrieved from the Roles table).
2. **Rows:** Display existing Permissions (retrieved from the Permissions table, populated by your sync script).
3. **Cells:** Checkboxes. Checking a box creates an entry in the Role_Permissions table.

Visual Grouping:

To avoid an indigestible list of 200 permissions, use the prefix from your naming convention (users:, invoices:) to create accordion sections in your table (e.g., "User Management" Section, "Billing" Section).

Technical Architecture Summary

Layer	Responsibility	Action
Code (Back)	Definition	Declares product:delete via annotation on an API route.
Sync Script	Discovery	Scans code at startup, inserts product:delete into Permissions table.

Layer	Responsibility	Action
Admin UI	Configuration	Displays product:delete. Admin checks the box for the "Manager" role.
Database	Storage	Saves the association Role: Manager $\leftrightarrow$ Permission: product:delete.
Runtime (Back)	Security	Rejects the DELETE request if the user does not have the "Manager" role.
Runtime (Front)	Display	Hides the "Delete" button if the user lacks the permission.